

Vulnerability analysis of S7 PLCs: Manipulating the security mechanism

Henry Hui^{*}, Kieran McLaughlin, Sakir Sezer

Centre for Secure Information Technology, Queen's University Belfast, Belfast, BT3 9DT, UK

ARTICLE INFO

Keywords:

Programmable logic controllers
PLC
Industrial control systems
Cyber-physical systems security
Anti-replay mechanism

ABSTRACT

Programmable Logic Controllers (PLCs) are the point of interaction between the cyber and physical world, and thus have been the target of previous cyber-attacks that caused physical disruption. To understand the effectiveness of state-of-the-art security mechanisms built into these devices, this paper presents an in-depth analysis performed on the Siemens PLC environment, particularly the communication protocol known as S7CommPlus. This protocol enables communication between Siemens endpoints such as TIA Portal (the engineering software from the vendor), and PLCs like the S7-1211C, which has been used for experiments in the work. The analysis utilises the tools WinDbg and Scapy. The anti-replay mechanism, used in the protocol is investigated, including the identification of specific bytes necessary to craft valid network packets. Novel exploits, including the manipulation of cryptographic keys, are identified based on experimental analysis. Subsequently, exploits are demonstrated that enable the stealing of an existing communication session, denying the ability of an engineer to configure a PLC, making unauthorised changes to PLC states, and other potential violations of integrity and availability. The problems that lead to these exploits are also discussed and a number of potential mitigation strategies are proposed.

1. Introduction

Industrial control systems (ICS) underpin the operation critical infrastructure such as water and sewage treatment facilities, power grids, and transport, as well as other infrastructure such as manufacturing processes and building management systems. Traditionally, ICS were separated from the outside world by an “air-gap” and cyber security was not a concern. However, increased connectivity, enabled by modern TCP/IP and Ethernet technologies, has led to a growing attack surface. An ICS will usually comprise various subsystems and components, e.g. Supervisory Control and Data Acquisition (SCADA) systems, Distributed Control Systems (DCS), Human Machine Interfaces (HMIs), Remote Terminal Units (RTUs), and Programmable Logic Controllers (PLCs).

Since PLCs are the point of interaction between the cyber and physical worlds, they have been the prime target in cyber-attacks. The most well-known attack targeting PLCs, Stuxnet, caused physical damage to the targeted ICS. The lack of security in these systems has received increased attention since Stuxnet, with industry introducing new cyber security aspects to PLCs. However, the effectiveness of these mitigations

is not widely studied in literature and there is a lack of analysis regarding the security of newer PLCs, firmware, and the programming environment. Therefore, to help improve the security of the PLC environment, a vulnerability analysis has been performed and is hereby presented. The interaction between three of the major elements in the PLC ecosystem (the proprietary software, the PLC, and the communication protocol), are the focus of this work. The presented analysis involved manual investigation and reverse engineering of the software and communication protocols used by PLCs to explore how an attacker might compromise a modern PLC environment. Siemens is one of the leading vendors in the world, and the modern Siemens S7-1211C controller was investigated for this analysis, and this paper presents details of potential exploits of the communication protocol by the S7 PLC.

1.1. Motivation and contribution

In similar, but more mature and well-established, fields of cyber security research there are numerous compelling contributions that are dedicated to vulnerability analysis. Such research aims to better

CCS CONCEPTS: • Security and privacy~Cryptanalysis and other attacks • Security and privacy~Systems security • Networks~Network protocols • Computer systems organization~Embedded and cyber-physical systems • **Hardware**

^{*} Corresponding author.

E-mail address: hhui01@qub.ac.uk (H. Hui).

<https://doi.org/10.1016/j.ijcip.2021.100470>

Received 4 September 2019; Received in revised form 13 August 2021; Accepted 27 August 2021

Available online 10 September 2021

1874-5482/© 2021 Elsevier B.V. All rights reserved.

understand, classify and document vulnerabilities with a goal of improving the long-term security in the targeted technology domain. Vulnerability analysis of cryptographic technology and specific implementations is well-established, and examples in the field of ICS include the discovery of a security weakness in a smart grid key distribution scheme [1]. Another established domain is analysis of communication protocols, where examples include the discovery of vulnerabilities and viable attacks against MAVLink, a communication protocol for unmanned aerial vehicles [2]. In the area of location or proximity based sensor applications, [3] performed a vulnerability analysis of an electronic attendance system and presented a signal spoofing attack based on the exploit discovered. In comparison, the security of the PLC ecosystem has not received similar levels of attention or research rigour. Most of the publications described later, in Section II, relate to PLC vulnerabilities and exploits based on outdated controllers and outdated PLC firmware, or lack detail in the fewer cases where newer technologies are investigated, e.g. [4] and [5]. Currently there is a lack of investigation into exploits against the latest PLC communication mechanisms, where security measures have been integrated. Although publications about older PLCs are still relevant to some degree, due to the long life-cycle of ICS components, it is crucial for the research community to identify emerging threats against newer controllers. Consequently, the presented work aims to help the ongoing process of improving security mechanisms, by investigating possible vulnerabilities in the most recent Siemens protocol, adopted with firmware version 4. Until now, no published papers have provided specific detail on exploiting this version 4 environment. Indeed much of the relevant work has been published via conferences or non-peer-reviewed (although well-known) channels such as hacker conventions. Consequently, one of the prime motivations for the work presented here is to provide a detailed and rigorous analysis, focusing on the PLC ecosystem, with the intention of improving the discourse and robustness around this particularly important domain for cyber security research and industry.

The presented work shows how an attacker could obtain sufficient knowledge of the S7 software, by using WinDbg to discover vulnerabilities that can be exploited to maliciously manipulate a PLC. The scope of this work is on investigating a method to initiate a session to a S7-1200 PLC and to identify potential exploits of the communication and anti-replay mechanism between TIA Portal and a PLC. The anti-replay mechanism is used in the newer version of the Siemens proprietary protocol and consists of a challenge, a response and integrity checks in the exchanged packets. As the investigated protocol is responsible for transferring the logic and configuration to PLCs, being able to initiate a session using this protocol would mean a PLCs, and therefore physical processes, could potentially be manipulated by attackers. In particular, this work provides a detailed vulnerability analysis based on the v4.1 firmware and TIA Portal v14, and the result are tested on firmware v4.2, beyond that presented in the closest work [6]. This work also clarifies some incorrect information provided by the related work. After analysing the protocol, some recommendations for immediate response to improve the security of the protocol are given. At the time of writing, some additional research emerged by Biham [7], which also investigates the Siemens proprietary protocol, but focuses on S7-1500 PLCs rather than S7-1200 PLCs. The differences between the discoveries, especially around detail of the anti-replay mechanism, will be presented in depth later.

The paper is organised as follows: Section II provides background information, including related work regarding vulnerabilities of PLCs. Section III will provide details on the Siemens PLCs ecosystem. Section IV describes the experiments and discoveries about how an attacker may compromise S7CommPlus communications. A discussion about the vulnerabilities and mitigation recommendations are offered in Section V.

2. Background

2.1. Stuxnet

The Stuxnet attack in 2010 demonstrated how compromised PLCs could cause physical impact [8,9]. The worm first propagated from a removable drive inside a uranium enrichment plant in Iran and infected a machine in the “air-gapped” network. The worm then propagated through the network by using four zero-day vulnerabilities and digital signatures from two legitimate companies. Apart from the sophisticated infection and propagation mechanisms used by Stuxnet, a key aspect from an ICS viewpoint was that it only targeted a few specific PLC models, specifically the Siemens S7 315 and 417, which were used to control the centrifuges for the enrichment process. Stuxnet determined whether a target was a Siemens PLC by checking the model number, configuration details, and downloading program code from the device. If the check failed, Stuxnet would not perform any further attack, possibly to avoid its actions being detected. Otherwise, it would proceed to update the main cycle block of the PLC with rogue code to increase the rotating frequency of connected centrifuges to damaging levels, while deceiving the operator by injecting false data to the HMI. The attack changed how the industry viewed security, and significant research began on this topic.

2.2. Programmable logic controllers

Generally, a PLC consists of several digital and analogue inputs and outputs, a processing unit, a memory module and an interface to communicate with other devices in the network. These controllers run a custom program which contains at least one organisation block as the main logic. Each of the blocks will be executed sequentially in a cycle. Proprietary vendor software is usually required to program or configure a PLC. Devices in a process control network may communicate with PLCs using serial connections or more recently by Ethernet, via protocols based on TCP/IP. While the protocols used during normal operations can be either openly standardised (e.g. Modbus TCP) or vendor specific, those used for transmission of custom PLC programs and other configuration details are usually proprietary. Although most proprietary software supports programming languages specified by the IEC61131-3 standard [10], there is no current standard on how the communication protocol should be implemented. Therefore, the reliability and the security of these proprietary protocols depends on the implementation provided by individual vendors. A more detailed description of Siemens PLCs and programming software is provided in Section III.

2.3. Related research

There are researches that are targeted on vulnerability analysis on either relevant device used or applied cryptology proposed on the specific field of research. [1] documented how a proposed cryptology scheme be exploited for an impersonation attack while [2] a widely used protocol in the field of unmanned aerial vehicles has been exploited to perform a denial-of-service attack.

In the field of ICS, Fig. 1. illustrates a timeline of several discovered exploits (labelled above the timeline), as well as the release date of the targeted PLCs and specific protocol versions (shown below the timeline). The details of each of these PLC-specific vulnerability discoveries are now discussed.

Beresford demonstrated a number of attacks on the S7-1200 PLC [11], e.g. replay attack, authentication bypass, starting or stopping a PLC, etc. The work was based on firmware v2 of the PLC, which includes no security mechanism in the communication protocol. However, the experiments provided the research community with an initial insight into vulnerabilities of this nature in relatively modern PLC devices.

“PLC inject” [12] exploits the S7Comm protocol (the last generation of the Siemens protocol, on which an anti-replay mechanism is not

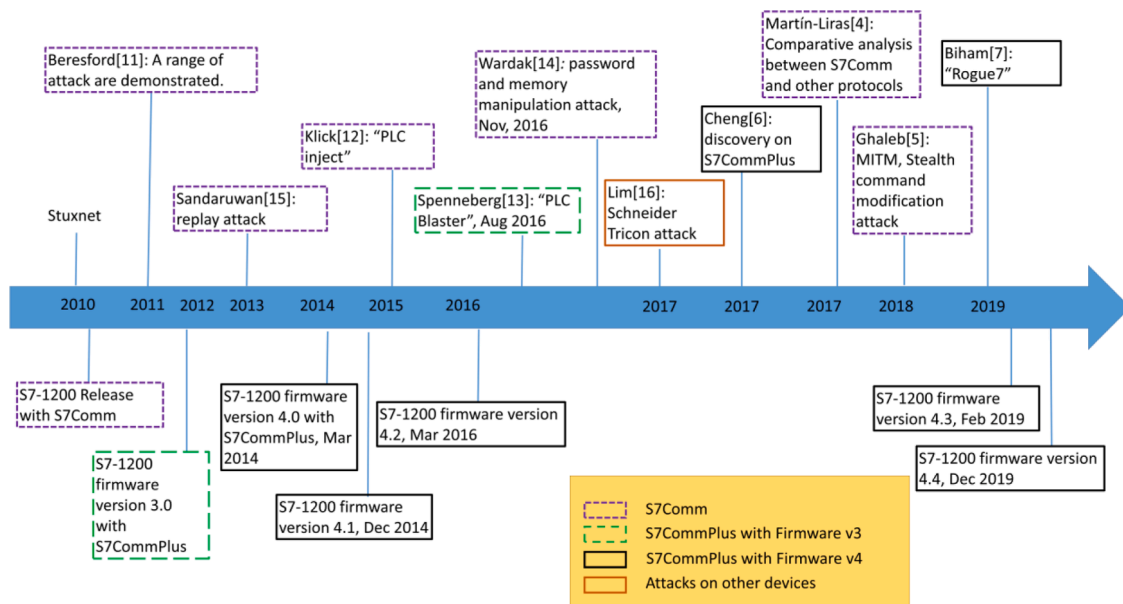


Fig. 1. Timeline showing exploit discoveries (top) and S7 protocol release dates (bottom).

included) and has the ability to add codes to the main program cycle of a PLC without interruption of services. It was also proven that it was possible to introduce a network proxy on a Siemens S7-300 PLC as a backdoor to the process control network.

A worm-like replay attack, which can spread amongst PLCs, called PLC Blaster, was developed and demonstrated on Siemens S7-1200 PLCs by Spenneberg et al. in 2016 [13]. The worm exploited vulnerabilities of the Siemens proprietary S7CommPlus protocol – specifically a newer version of the protocol with an anti-replay mechanism. The protocol used in the S7-1200 PLCs, with firmware version 3, was released after vulnerabilities exploited by Stuxnet became known. The protocol is responsible for communicating information from the engineering software to program and control PLCs. Spenneberg et al. demonstrated a malicious program, created using the Siemens programming software, which is uploaded to a PLC using an Ethernet cable. The malicious upload also includes the bytes of the network packets necessary to program additional PLCs. The compromised PLC then automatically scans the network and connects to other PLCs and attempts to upload the rogue code into them. The affected PLC will be restarted during the infection, which could attract attention. Siemens has since updated the firmware version of affected PLCs to firmware version 4, which mitigates this vulnerability.

Password protection is usually provided by vendors to secure PLCs against unauthorised action to be performed. However, it has been shown that password stealing, password reset and memory control attacks [14] are possible in Siemens S7-400 PLCs via replay attacks. Moreover, PLC passwords are often simply not used in practice. An older publication [15] also demonstrated the possibility of replay attack, man-in-the-middle attack and authentication bypass attack. Both [14] and [15] exploit the lack of security in the communication protocol in older Siemens PLCs, prior to 2015.

Lim et al. [16] documented an investigation of the Schneider Tricon PLC in 2017. Using reverse engineering techniques, the structure of the Tricon communication protocol was identified. Attacks were demonstrated that allowed changing the control logic and the ability to cause immediate failure to the Tricon PLC. The latter attack requires a manual reset of the PLC to recover, which could cause significant problems in a real environment as availability is often essential.

A more recent publication in 2017 proposed a way to decompile and map byte codes into PLC program instructions [17]. This could allow an attacker to understand the details and processes that are running.

However, the PLC involved in the work is the S7-200 series PLC, which was listed by Siemens as phase-out in 2013 [18].

In 2017, experiments presented by L. Cheng et al. provided some information about the new anti-replay mechanism used in the S7CommPlus protocol for firmware version 4 [6]. The TIA Portal software was examined by using reverse engineering techniques to investigate the communication between the PLC and TIA Portal. A brief overview of how the anti-replay bytes were generated was presented. This is a potential exploit that could be used to replay packets to a PLC. This is one of the closest work to the experiments presented here, although its scope was confined only to the identification of an exploit. Furthermore, the information regarding the algorithm used in the anti-replay mechanism is partly inaccurate and the construction of the anti-replay bytes is not discussed. Without such information it is not possible to reproduce the work, or for vendors and operators to develop security mitigations. This paper will provide information regarding the above issues.

A comparative analysis between three proprietary protocols used by PLCs was presented by L. Martin-Liras [4] in 2017. The three protocols involved are S7Comm protocol (used by S7 PLCs), UMAS protocol (used by Modicon PLCs) and OptoComm protocol (used by Opto PLCs). Research was done on existing information regarding the protocol vulnerabilities. The analysis also investigated the possibility on performing various attacks on PLCs, including DoS, firmware alternation, and changing user program etc. A newer publication in 2018 by L. Ghaleb [5] documented an investigation on the vulnerabilities of the S7 protocol specifically, on which Man-in-the-Middle attack, command modification and replay attack were demonstrated on a S7-400 PLC that uses the S7Comm protocol.

In 2018 the authors published preliminary work related to the research now presented [19]. A general network attack, based on ARP poisoning, was proven possible in the S7-1211C PLC (where firmware version 4 was used) and TIA Portal. The authors also found that a 7-byte "S7-ACK" packet, "03 00 00 07 02 f0 00", can be exploited for session stealing and DoS attack. Section V will provide relevant details of this previous investigation in the context of the novel work now presented.

In 2019, further investigation of the Siemens S7 protocols emerged by Biham et al. [7] and attacks on reprogramming S7 PLCs, which were named Rogue7, were demonstrated. The Siemens PLC ecosystem was investigated using reverse engineering techniques, targeting the S7-1500 series PLCs rather than S7-1200 PLCs, as in this work. While

this parallel research by Biham shows some overlap with our own, particularly in identification of the function message integrity checks, gaps remain regarding details in the analysis of the protocols and around the encryption and anti-replay mechanism. Given the proprietary nature of the industrial protocol, reproduction of the reverse engineering work requires further low-level information on these issues than is currently available in the literature, including the tabulation hashing used (a type of hashing that is constructed from lookup table and exclusive or operations), the computational flow of the security mechanism, byte level manipulation, pseudo code for an 8-byte value critical to the S7 function integrity check, and so on. The presented experiments therefore aim to provide substantially more evidence, detail, and insight than has been published by [6, 7] and elsewhere.

2.4. Mitigation for PLC exploits

As highlighted in the previous section, various updates have been made to the relevant Siemens firmware over recent years to mitigate specific vulnerabilities, once discovered. This proactive approach is to be welcomed. Other approaches to mitigation have also been proposed in the research community. One important aspect on protecting PLCs from attack is to monitor and secure the custom PLC logic. A few novel architectures have been proposed: Trusted safety verifier (TSV) [20], Secure System Simplex Architecture (S3A) [21], and Controller (C2) [22]. Each of these measures aims to provide security, with a trade-off between timely alerts and the complexity the architecture can handle. In contrast to these three “bump-in-the-wire” approaches, the Shadow Security Unit [23] monitors and correlates events in parallel to the PLC. It is part of a framework [24] on securing SCADA systems and has the ability to locally correlate events or forward the information to central monitoring systems. All of these approaches require a device to be connected to the network, for which the cost and complexity will scale according to the size of the network.

Another aspect to protecting the PLC environment is to utilise network security measures. Using network telemetry analysis and choosing a set of factors that are relatively difficult for attackers to control [25], an Intrusion Detection Systems (IDS) has been demonstrated to be able to differentiate malicious traffic from the legitimate SCADA operations. There are also other anomaly-based IDS approaches, such as [26] and [27] that involve the monitoring network traffic associated with the Siemens S7 communication protocol. It is anticipated that the discoveries presented in this paper, which include new network-based exploits, can contribute to improving network-based monitoring and detection approaches for the PLC network environment.

2.5. Tools and methodology

This work adopts a methodology using principles similar to the analysis by Kim, which carried out an analysis of a location and proximity based application [3]. Kim started the analysis with a manual investigation and custom software to identify patterns in the transferred message, for example the byte position of specific values. Further experiments are then performed utilising the discovered information towards finding a way to exploit the mechanism. In this work, the method applied during this investigation rely upon detailed manual analysis and reverse engineering techniques. During manual analysis, Wireshark [28] and custom python scripts are used to identify patterns in various sessions of the S7 communication. Additionally, Scapy [29], a python packet manipulation tool, is used to replay selected and crafted packets to the PLC and the corresponding replies are studied. In parallel, WinDbg [30] is used to perform reverse engineering of the Siemens software. WinDbg is software distributed by Microsoft for kernel debugging and to examine CPU registers and memories. The focus of this analysis is on the Total Integration Automation Portal (TIA Portal), and a detail discussion about this and other aspects of the PLC environment now follows.

3. Siemens PLC ecosystem

3.1. Siemens PLCs and TIA portal

Modern PLCs available from Siemens include the S7-1500 and the S7-1200 models with firmware version 4 and above, which can operate using the latest S7CommPlus protocol. An anti-replay mechanism is built into the communication protocol to defend against replay attacks. Table 1 summarises the communication protocol of S7-1200 and S7-1500 PLCs. There are two version of S7CommPlus protocol, where version 1 includes an anti-replay byte for security, while version 2 is protected with full anti-replay mechanism and function integrity check. For the rest of this work, when mentioning the S7CommPlus protocol, we refer to the latest version of the protocol, i.e. version 2.

There are numerous older PLCs still being deployed, including the S7-300 series, S7-400 series, and S7-1200 PLCs with older firmware. These PLCs are prone to the vulnerabilities documented in Section II. This work targets the Siemens S7-1200 series PLCs, where there have been several iterations of firmware version, as show in Fig. 1.

As mentioned in Section 2.3, there was no built-in security mechanism prior to firmware v3, which uses the predecessor of the S7CommPlus protocol, S7Comm, for communication. A security mechanism was introduced in firmware v3, which involves only one anti-replay byte per packet [12]. Firmware v4 was then introduced, along with an anti-replay mechanism in an attempt to rectify the problem of insufficient security provided by the previously introduced anti-replay byte. This work presents the results of the vulnerability analysis performed on this newly introduced security mechanism. The PLC used in this work is the S7-1211C, with the results verified using both v4.1.3 and v 4.2.3 firmware versions specifically.

TIA Portal is Siemens' proprietary automation software environment, used to program and adjust the settings of connected PLCs, e.g. configuring IP addresses, run or stop a PLC, or modify values of PLC inputs and outputs. TIA Portal v14 is used in this work.

3.2. S7Commplus protocol

The S7CommPlus protocol facilitates the transfer of critical operational and configuration information, such as PLC logic, diagnostic information, configuration details, and data block values between the PLCs and engineering software. Fig. 2 shows the dissected protocol stack of a packet carrying S7CommPlus data viewed in Wireshark. TCP port 102 is used for all S7 communications. The S7CommPlus protocol runs on ISO on TCP (TPKT) [31], and Connection oriented Transport Protocol (COTP). If an operator initialises a connection to a PLC from TIA Portal, e.g. pressing the “Go online” button in TIA portal as shown in Fig. 3, the following steps happen, as depicted by Fig. 4:

- 1 TIA Portal broadcasts a Profinet Discovery and Basic Configuration Protocol (PN-DCP) “Identify All” packet to the network.

Table 1
S7-1200 and S7-1500 PLC protocol and security comparison.

	Communication	Security	Addressed By Research
S7-1500 prior to v1.8	S7CommPlus version 1	With one anti-replay byte	–
S7-1500 v1.8 and above*	S7CommPlus version 2	With full anti-replay mechanism and function integrity check	[7]
S7-1200 v3	S7CommPlus version 1	With one anti-replay byte	[4, 5, 11, 14, 15]
S7-1200 v4 and above*	S7CommPlus version 2	With full anti-replay mechanism and function integrity check	[6] and this work

*TIA portal V14 or above is needed for the use of the use of S7CommPlus protocol with anti-replay mechanism built in.


```

Frame 21: 293 bytes on wire (2344 bits), 293 bytes captured (2344 bits)
Ethernet II, Src: [REDACTED], Dst: [REDACTED]
Internet Protocol Version 4, Src: 168.168.1.2, Dst: 168.168.1.121
Transmission Control Protocol, Src Port: 2818, Dst Port: 102, Seq: 23, Ack: 23, Len: 239
TPKT, Version: 3, Length: 239
ISO 8073/X.224 COTP Connection-Oriented Transport Protocol
S7 Communication Plus

```

Fig. 2. Protocol Stack of S7CommPlus.



Fig. 3. "Go online" in TIA Portal.

- 2 All PLCs or devices reply to TIA Portal with a PN-DCP "Identify OK" packet.
- 3 TIA Portal initialises a TCP handshake with the PLC, and the PLC will reply.
- 4 TIA Portal and PLC exchange COTP connection information.
- 5 TIA Portal sends the first S7 packet (also see Fig. 5).
- 6 PLC replies with a packet containing a 1-byte and a 20-byte anti-replay challenge (S7 Challenge, Fig. 6).

- 7 TIA Portal replies a packet containing an anti-replay byte and a 132-byte array, which is the anti-replay response (S7 response, Fig. 7).
- 8 TIA Portal sends packets with the action requested to the PLC, along with a 20-byte integrity check in every packet (Fig. 8)

Figs. 5–7 show TIA Portal and the PLC exchanging S7 request, challenge and response packets, where special bytes are involved in the process. The numbered labels, e.g. ① in Fig. 5, will be referred to in later sections of this paper. If any S7CommPlus packets do not include correct anti-replay bytes or integrity check values, the other end of the connection will send a TCP reset packet and the session will be terminated.

Once the button "Go online" in Fig. 3 is pressed in TIA portal, a session will be initiated using the S7 protocol, as in Fig. 4. The operator can upload custom programs, diagnose problems related to the PLC,

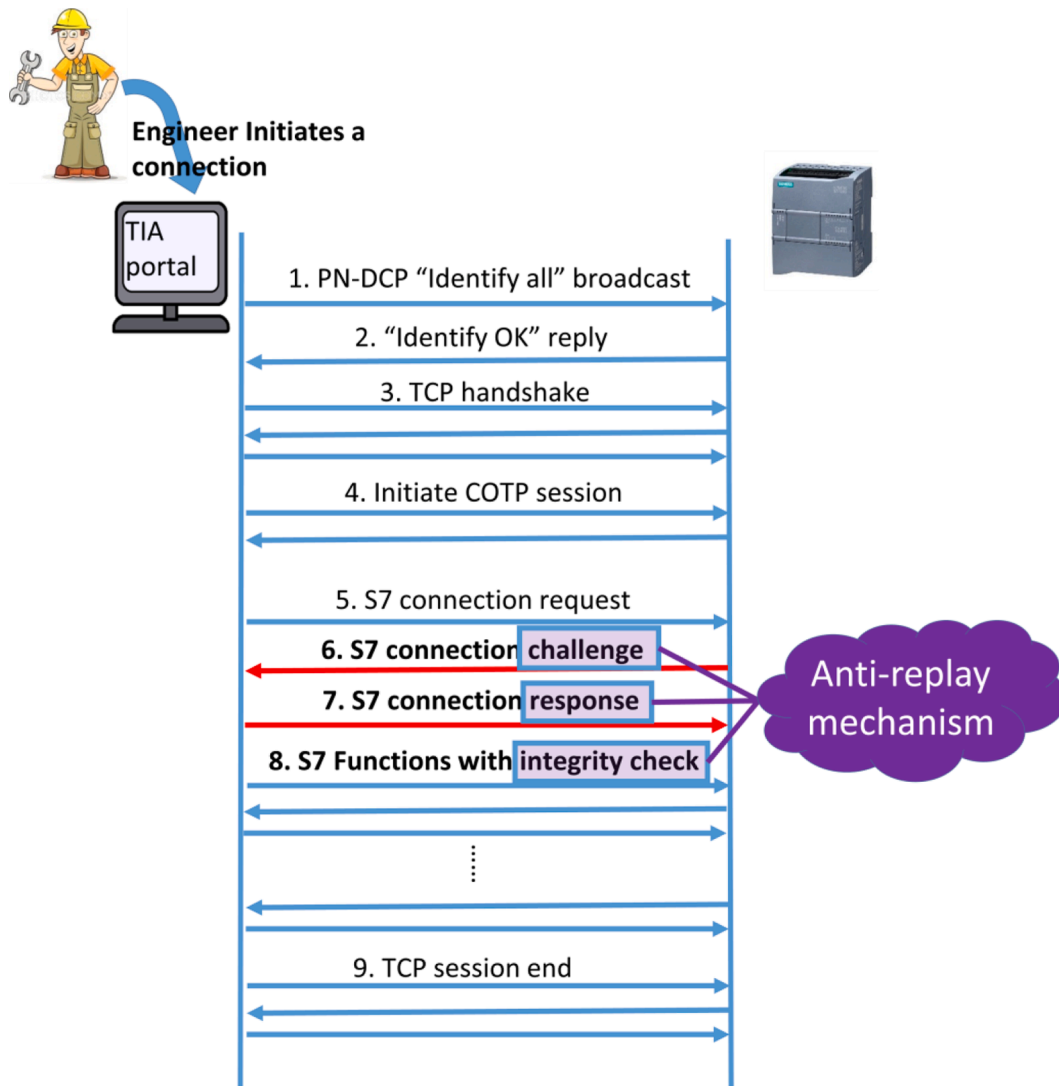


Fig. 4. S7CommPlus session and the anti-replay mechanism.

```

00000016 03 00 00 f2 02 f0 80 72 01 00 e3 31 00 00 04 ca .....r ...1...
00000026 00 00 00 01 00 00 01 20 36 00 00 01 1d 00 04 00 ..... 6.....
00000036 00 00 00 00 a1 00 00 00 d3 82 1f 00 00 a3 81 69 .....i
00000046 00 15 15 53 65 72 76 65 72 53 65 73 73 69 6f 66 ①...Serve rSession
00000056 5f 31 43 39 43 33 38 31 a3 82 21 00 15 3c 31 3a ②...1C9C381...<1:
00000066 3a 3a 36 2e 30 3a 3a 49 6e 74 65 6c 28 52 29 20 ::6.0::I ntel(R)
00000076 38 32 35 37 38 44 4d 20 47 69 67 61 62 69 74 20 82578DM Gigabit
00000086 4e 65 74 77 6f 72 6b 20 43 6f 6e 6e 65 63 74 69 Network Connecti
00000096 6f 6e 2e 54 43 50 49 50 2e 31 a3 82 28 00 15 00 on.T...1..(....
000000A6 a3 82 29 00 15 00 a3 82 2a 00 15 13 41 44 4d 49 ..)....*...ADMI
000000B6 4e 32 2d 50 43 5f 31 36 32 39 31 33 39 38 31 a3 N2-PC...16 2913981.
000000C6 82 2b 00 04 01 a3 82 2c 00 12 01 c9 c3 82 63 82 .+....., .....
000000D6 2d 00 15 00 a1 00 00 00 d3 81 7f 00 00 83 81 69 ~.....i
000000E6 00 15 15 53 75 62 73 63 72 69 70 74 69 6f 6e 43 ...Subsc riptionC
000000F6 6f 6e 74 61 69 6e 65 72 a2 a2 00 00 00 00 72 01 ontainer .....r.
00000106 00 00

```

Fig. 5. S7CommPlus (connection) request (from TIA portal).

```

00000016 03 00 00 c5 02 f0 80 72 01 00 b6 32 00 00 04 ca .....r ...2...
00000026 00 00 00 01 36 00 02 87 08 87 3d a1 00 00 01 20 ....6... ..=...
00000036 82 1f 00 00 a3 81 69 00 15 02 30 31 a3 82 2b 00 .....i. ....01.+..
00000046 04 82 80 80 00 a3 82 2d 00 15 10 4f 4d 53 50 .....-...OMSP
00000056 2e 52 45 4c 2e 37 30 37 30 2e 31 34 a3 82 2f 10 .REL.707 0.14../.
00000066 02 14 93 de 09 e0 47 bd df 3a ff 88 ab 77 53 e2 .....G. ....wS.
00000076 6e 16 a9 9e cf 0e 11 a3 82 32 00 17 00 00 01 3a 82 .j..... 2.....:
00000086 3b 00 04 83 40 82 3c 00 04 83 00 82 3d 00 04 84 ;...@.<. ....=...
00000096 80 c1 40 82 3e 00 04 84 80 c1 40 82 3f 00 15 1b ..@.>... ..@.?...
000000A6 31 3b 36 45 53 37 20 32 31 31 2d 31 41 45 34 30 1;6ES7 2 11-1AE40
000000B6 2d 30 58 42 30 20 3b 56 34 2e 31 82 40 00 15 05 -0XB0 ;V 4.1.@...
000000C6 32 3b 33 31 30 82 41 00 03 00 03 00 a2 00 00 00 2;310.A. ....
000000D6 00 72 01 00 00

```

Fig. 6. S7 challenge packet (from PLC).

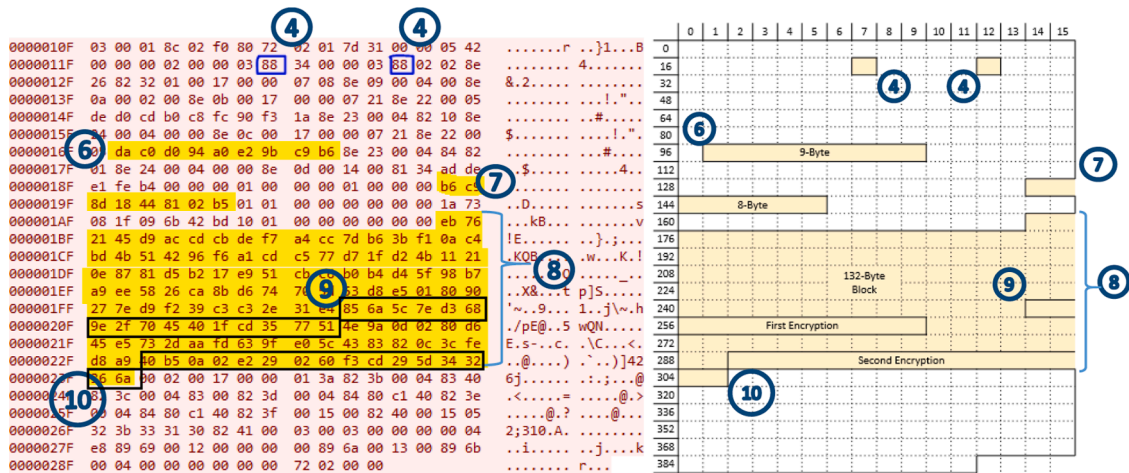


Fig. 7. S7CommPlus response packet (from TIA portal) (left: Traffic capture, right: Representation of byte position).

```

000002A2 03 00 00 65 02 f0 80 72 03 00 5a 75 38 d8 cb .....r ...V u8..
000002B2 03 f4 cc ec dd 6d 20 d8 56 4d 47 33 5c f9 e3 39 .....m. VWG.V..9
000002C2 d8 c2 57 8d 97 35 81 ae 52 0d 00 53 10 00 05 ...g...S...R...e1..
000002D2 56 00 00 03 00 00 03 88 34 00 00 00 31 20 04 ..)....4...1..
000002E2 01 9d 29 00 00 04 e8 89 69 00 12 00 00 00 89 ..)....1.....
000002F2 6a 00 13 00 89 6b 00 04 00 00 01 01 00 00 00 j.....k.....
00000302 00 72 03 00 00

```

Fig. 8. S7CommPlus function packet (from TIA portal).

view real-time data from the PLC data blocks, configure communication between PLCs and other devices, etc. During an online period with the S7-1211C PLC, several packets are sent to TIA Portal during idle time

specifying details and live status of the PLC, e.g. cycle time, memory usage, etc. Previous works [6,7], identified some bytes that are related to the anti-replay mechanism.

3.2.1. Anti-Replay byte

The S7CommPlus protocol utilises a 1-byte value in the anti-replay mechanism, which has been used since S7-1200 firmware version 3. It was first identified and published in 2016 [12]. When TIA Portal initiates a connection to a PLC, the PLC sends a challenge byte in the range 0×06 to $0 \times 7f$. TIA Portal will reply to the PLC with a response which is calculated by adding the value 0×80 to the challenge. For example, if the PLC challenge is 0×08 , the TIA Portal response would be $0 \times 08 + 0$

$\times 80 = 0 \times 88$, as shown at label ④ in Fig. 6 and Fig. 7. The challenge is in the 25th byte position and the response is located in the 24th and 29th byte of the respective packets. In PLC firmware version 3, subsequent function packets, following the S7 response packet, must include the same anti-replay byte in the 24th byte position; in firmware version 4, it is found to be the 57th byte or 62nd byte of the relevant packet.

3.2.2. Encryption in the response packet

Since firmware version 4, the response packet from TIA Portal has to include several bytes in addition to the single anti-replay byte discussed above. In 2017 Cheng [6] discovered two 16-byte encryptions involved in the packet, where the second encryption is dependant on the first encryption. The two 16-byte values start in the 235th and 291st byte of the S7 response packet, as shown by labels ⑨ and ⑩ in Fig. 7. The first encryption is an XOR operation while the second encryption is generated by a more complex algorithm. Part of this dependency is confirmed by this work and later by Biham [7].

3.2.3. Function packet “Encryption”

After the response packet is sent to the PLC, TIA Portal will begin sending bytes related to the functions of TIA Portal, which will be named “function packets” for the rest of this paper. All of these packets have to include a 32-byte value of “encryption”, as shown by label ⑪ in Fig. 8. This “encryption” is discovered to be an integrity check facilitated by Hashed-Based Message Authentication Code (HMAC) and is related to the anti-replay byte. Cheng proposed the possibility of starting and stopping a PLC by exploiting the protocol, but not however provide details describing the protocol vulnerabilities, and simply notes the function packet integrity check being an “encryption”. Subsequently, Biham identified the underlying mechanism used in the S7 Protocol as being a HMAC and discovered the detail of the key for the HMAC is exchanged through Elliptic-Curve El-Gamal-like key exchange. However, specific protocol mechanisms were not described, e.g. each field of the anti-replay response are loosely identified with pseudo code, while the algorithmic execution detail is missing. Similarly in the function packet HMAC computation, two sets of HMACs with two different keys are discovered that were not previously identified. Therefore, research addressing these gaps are investigated and presented further in Section IV, alongside related algorithms, where a detailed analysis of the protocol and vulnerabilities are presented to provide a more detailed view of the mechanisms supporting encryption.

Table 2. provided a summary of the numbered bytes from Fig. 5–8. Some of these fields and bytes will be referenced in the next section, where the mechanisms of the S7CommPlus protocol will be dissected in detail.

4. Vulnerability analysis experiment

Experiments were carried out in a testbed comprising an engineering station running TIA Portal v14, two S7-1211C PLCs with firmware versions v4.1.3 and v 4.2.3, the latest at time of writing, and a “compromised” machine, all connected to a shared LAN via a switch.

4.1. Analysis of S7CommPlus protocol

A manual analysis was conducted following the methods outlined in Section II.E, using tools such as Scapy and WinDbg to examine the communication between TIA Portal and the PLCs during a number of different communication sessions. In the first experiment, by manually inspecting the packets, including byte manipulation using Scapy, the bytes labelled in Figs. 5–8 were discovered to serve a number of specific purposes, which will now be explained. As a reminder, Figs. 5–7 show TIA Portal and the PLC exchanging S7 request, challenge and response packets, while Fig. 8 shows a subsequent “function packet”. Also see Fig. 4 which depicts the overall exchanges in a session.

Fig. 5, label ① indicates a server session number that is generated by

Table 2

Description of the numbered byte in Fig. 5–8.

Field Number	Description	Purpose
①	Server Session number in ASCII	Identifier for the communication session.
②	PC Session number	Identifier for the TIA Portal session.
③	Server Session number in hex	Identifier for the communication session.
④	Anti-replay byte	Anti-replay byte first found from previous firmware version.
⑤	S7 Connection Challenge	Used in key generation for S7 response and integrity check.
⑥	7-byte string	Part of the S7 response for key generation that can be manipulated.
⑦	9-byte string	Part of the S7 response for key generation that can be manipulated.
⑧	132-byte block	One of the main parts of the S7 response, which includes information for both encryptions and information on generating them.
⑨	First encryption	One of the main parts of the anti-replay mechanism that is checked by PLCs when a session is initiated
⑩	Second encryption	One of the main parts of the anti-replay mechanism that is checked by PLCs when a session is initiated. It is dependant on the first encryption,
⑪	24-byte integrity check	Integrity check bytes that are compiled for every packet containing S7 data.

TIA Portal every time a communication session is started. However, this number can be reused and is not checked by the PLC. At label ② is another session number, or “PC” session number, that is generated every time TIA Portal is opened (i.e. the first time TIA Portal is opened or after a computer restart, hence the name “PC” session number). This number can also be reused. Label ③ is the value of the server session repeated directly in hex. Label ⑤ in Fig. 7, in contrast with other bytes in the same packet, is significantly different for every session. It appears most likely to be generated by a hash or pseudo random function. Label ⑥, ⑦, and ⑧ in Fig. 7 are 9-byte, 8-byte and 132-byte blocks respectively. These three blocks are checked by the PLC and a TCP packet with a reset flag will be immediately sent by the PLC if the blocks of bytes are not what the PLC expected, otherwise the communication continues. These bytes are generated by a specific algorithm, which will be introduced later in this section.

After the TIA Portal response, if the connection is accepted, an S7CommPlus packet is sent by the PLC. The S7CommPlus packet will contain information related to the functions provided by TIA Portal. Sophisticated attackers could identify these bytes and use this information to further exploit the protocol.

4.2. Analysis of TIA portal

In order to understand the algorithm that generates the anti-replay response, and to explore possible exploits, WinDbg was used to perform an analysis of TIA Portal. This section describes the step-by-step analysis process undertaken during the experiments. For experimental purposes, the function that executes a search for accessible devices in TIA Portal was used to generate S7CommPlus traffic. The outcomes discovered during the analysis are depicted in Figs. 9–15. Labels ④ to ⑥ in Fig. 9 will be referenced throughout this discussion. There are two main parts in Fig. 9: how the challenge array is processed (blue box) and the process hereafter named “Function B” (yellow box), which is where most of the bytes of the anti-replay mechanism are generated.

To support the analysis, several breakpoints were set during the communication between the software and the PLC. A process of manual analysis identified that byte array ⑤ changes significantly every time TIA Portal makes a request. This 20-byte block has no obvious

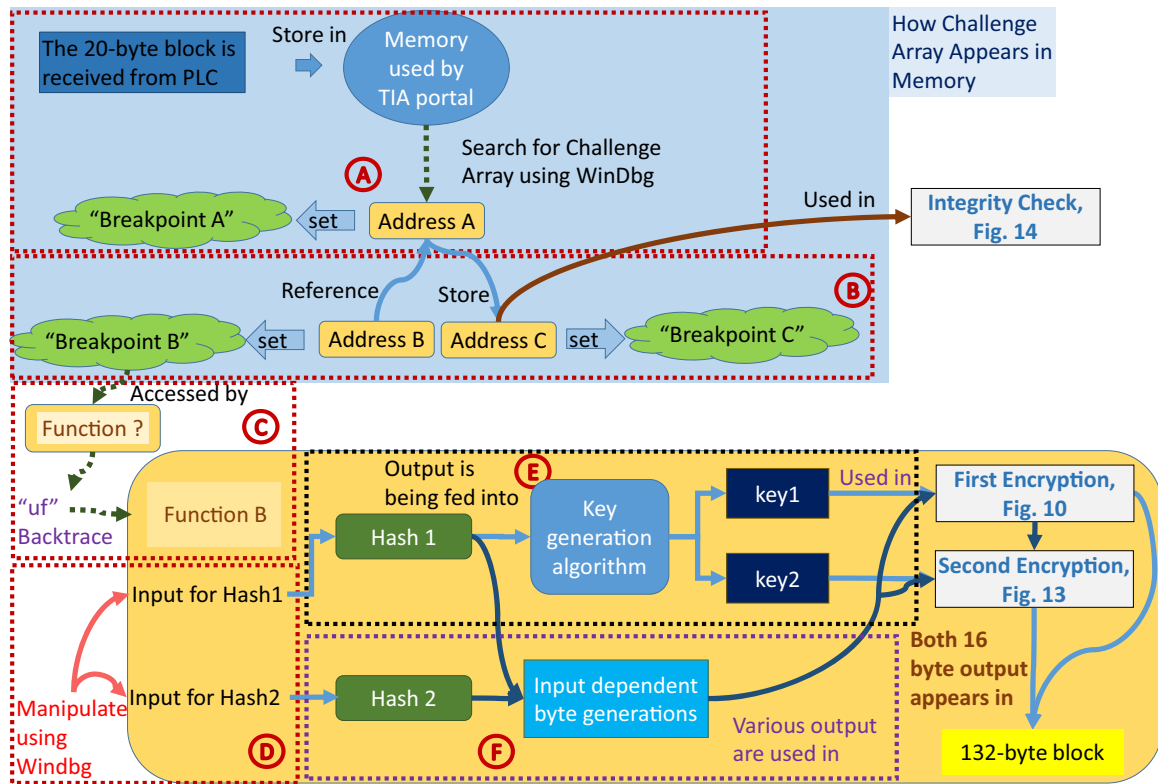


Fig. 9. Steps A to F, which are carried out to analyse the TIA Portal process. Blue box shows “Challenge Array”. Yellow box shows “Function B”.

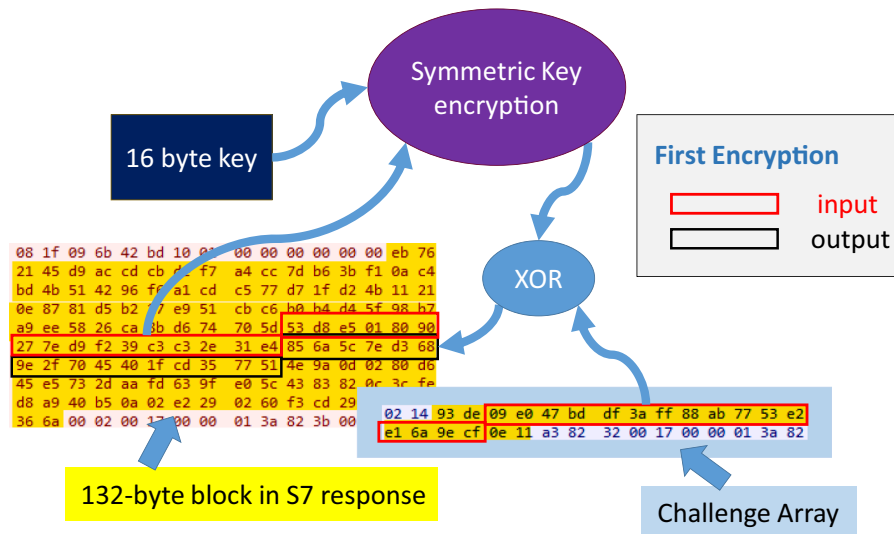


Fig. 10. First Encryption.

relationship with the payload of the connection request packet, as confirmed by sending the exact same connection request packet to the PLC using Scapy. From previous work such as [6] and [7], it is not obvious where to start the reverse engineering process, given limited information available on the S7CommPlus protocol. The first challenge for anyone carrying out this analysis is therefore identifying the part of the ecosystem that is susceptible to attack and the entry point of the analysis, on which they are given as follows. To start the analysis, the search function of TIA Portal is used once without any breakpoint and one complete session of communication is generated (ended by a TCP reset packet from the PLC). By manually initiating a break via WinDbg on the software, and then using the command “s” to search for the

20-byte from the PLC in the memory, the memory address containing this 20-byte block can be identified. This memory address could be located with only one “s” search, or often only the area storing the whole received S7 challenge packet is located. A further search has to be done by using an on-access breakpoint and to trace the specific address to which this 20-byte array is written. Once this address is identified, it does not change until TIA Portal has been restarted. As illustrated by label A, an on-access breakpoint, “breakpoint A”, is set on this address. After another S7 communication is initialised by using TIA Portal, it was found that breakpoint A was accessed by two different locations, both of which are functions that involves copying the 20-byte block to another address. The first function copies the address pointed by breakpoint A,


```

def finite_field_alike_incrementation (
    input: 16 byte input, byte_in
           7 by 16 byte value, Xp

    output: 256 by 16 byte array with the incremented bytes, value256)

    Xp = {2: int("00000000100000000000000010000000",16),
          4: int("0000000010000000010000000100000000",16),
          8: int("0000800400000000000000000000000001",16),
          16: int("0000000010000000000000000100000000",16),
          32: int("0000800500000000010000000100000001",16),
          64: int("0000800500000000000000000100000001",16),
          128: int("0000000010000000000000000100000001",16)}

    root = [2,4,8,16,32,64,128]
    value256[0] = 0
    value256[1] = byte_in
    for round in range(2,256):
        bin_str = to_binary_string(round)
        if round in root:
            value256[round] = (value256[round/2] * 2) ^ Xp[round]
            if value256[round] >= 2**128:
                value256[round] = value256[round][1:]
        else:
            temp = 0
            temp1 = 0
            for pos in bin_str:
                if pos == 0:
                    continue
                temp = value256[2**(len(bin_str)- pos -1)]
                temp1 ^= temp
                if temp1 >= 2**128:
                    temp1 = temp1[1:]
            value256[round] = temp1
    return value256

```

Fig. 11. "Finite field" incrementation algorithm.

while the second function copies the bytes to a specific address, as illustrated at label ⑥. Therefore to continue the investigation, two further on-access-breakpoints, breakpoint B and breakpoint C were set for each of the two identified addresses.

It was found that breakpoint B points to the address of the 3rd byte and breakpoint C stores the 3rd to 18th byte of the 20-byte array (label ⑤ in Fig. 6). This 16-byte value (bytes 3 to 18), or "challenge array", has been discovered to play an important role for the rest of the byte generation process. It has further been discovered that the two memory locations that breakpoint B and C point to are involved in generating bytes for the S7CommPlus response and function packet respectively.

It is at this point that the dll, "OMSp_core_managed.dll" first appears and the addresses pointed to by breakpoint A, B and C are within the range of this dll. During the analysis, it was confirmed that, this module, or dll, is where all the generation of the anti-replay bytes take place. The generation of the anti-replay bytes in the S7response packets and S7 function packets will be explained below.

4.2.1. S7 response packet from TIA portal

By following breakpoint B in a new S7 session, where the challenge array is accessed, "Function B", from the module "OMSp_core_managed" is identified by using the "uf" command in WinDbg, as shown at label ⑦. Note the "uf" command returns the whole function to which the address belongs. After further investigation by back tracking the call stack of Function B, it was confirmed to be where the S7CommPlus response packet is generated. The response packet can be divided in several parts, as detailed below.

4.2.1.1. Bytes that can be manipulated.

It was discovered in the

investigation that the hash algorithm SHA-256 is used at least twice. The inputs of these hashes are generated by the "CryptGenRandom" Windows API. The two hashes generated are used as part of the input to generate the bytes in the response packet. These bytes include: the 9-byte and 8-byte block, (⑥ and ⑦ respectively), the first 76 bytes of the 132-byte block ⑧, and the bytes between encryption 1, ⑨, and encryption 2, ⑩. As shown at ⑩, these byte blocks can be manipulated by changing the input of the hash function using WinDbg, for example setting the input to all zeros, and the output of these bytes will stay constant in every session. This can be a very powerful exploit, as once these bytes are generated, it allows packets to be generated without the need of either TIA portal or WinDbg. The PLC will accept these "crafted" packets as long as the other bytes were also generated by the appropriate algorithm.

Point ⑩ illustrates how one of the hashes has been identified as part of a process that generates two 16-byte keys, "Key 1" and "Key 2" in Fig. 9. These keys are subsequently used in two symmetric-key encryption processes, specifically AES-128, in the S7CommPlus response. The two processes will be referred to as "First Encryption" and "Second Encryption", as shown on the right of Fig. 9. Since the input of these hashes can be manipulated (see ⑩), the output hashes, and therefore the two keys, can be generated and will change according to the input of the algorithm. Hence, label ⑩ can be considered as a black-box. There are further values that change depending on the input of the hash algorithm, and these are generated by another set of functions. For the purposes of the presented analysis, the detailed operation of these functions does not need to be known, and they can therefore also be considered as a black-box, which is labelled ⑪ in Fig. 9. This will be discussed again in due course. It is currently believed to be impossible to

```

def a480 (
    input: 4 by 4 byte array input, b_input
           256 by 16 byte array with the incremented bytes, value256

    output: A 128-bit long byte string)

    tempout = 0
    for position in range(0,4):
        for element in range(0,4):
            #Read byte from input, starting from b_input[0][0], then b_input[1][0]...
            multiple = b_input[element][position]
            #lookup the corresponding value in the 255 multiplier (value255)
            temp = value255[multiple]+ "0"*(4-element)*0x20
            tempout ^= temp
        if position<3:
            ## shift all byte to the right by one byte in big endian
            ## and change back to little endian again
            jtemp = to_big_endian(tempout)
            jtemp = jtemp >> 0x8
            tempout = to_little_endian(jtemp)

    A = tempout[0:4]
    B = tempout[4:8]
    C = tempout[8:12]
    D = tempout[12:16]
    E = tempout[16:20]
    F = tempout[20:24]
    G = tempout[24:28]
    H = tempout[28:32]

    q = H >> 0x0d ^ H >> 0x11 ^ E ^ H
    t = F << 0x0d ^ F << 0x02
    u = G << 0x0d ^ G << 0x02
    v = H << 0x0d ^ H << 0x02

    output[0] = q << 0x0d ^ q << 0x02 ^ A ^ q
    output[1] = q >> 0x0d ^ q >> 0x11 ^ t ^ B ^ q ^ F
    output[2] = F >> 0x0d ^ F >> 0x11 ^ u ^ C ^ F ^ G
    output[3] = G >> 0x0d ^ G >> 0x11 ^ v ^ D ^ G ^ H

    return to_byte_string(output)

```

Fig. 12. Algorithm a480.

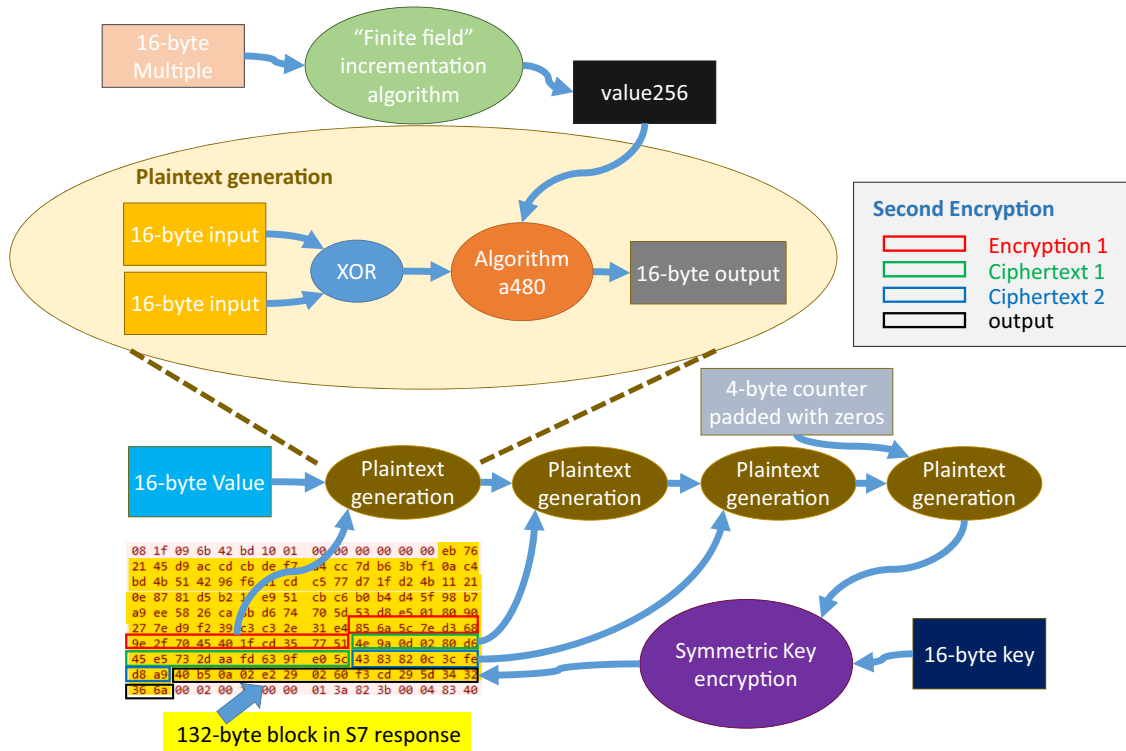


Fig. 13. Second Encryption.

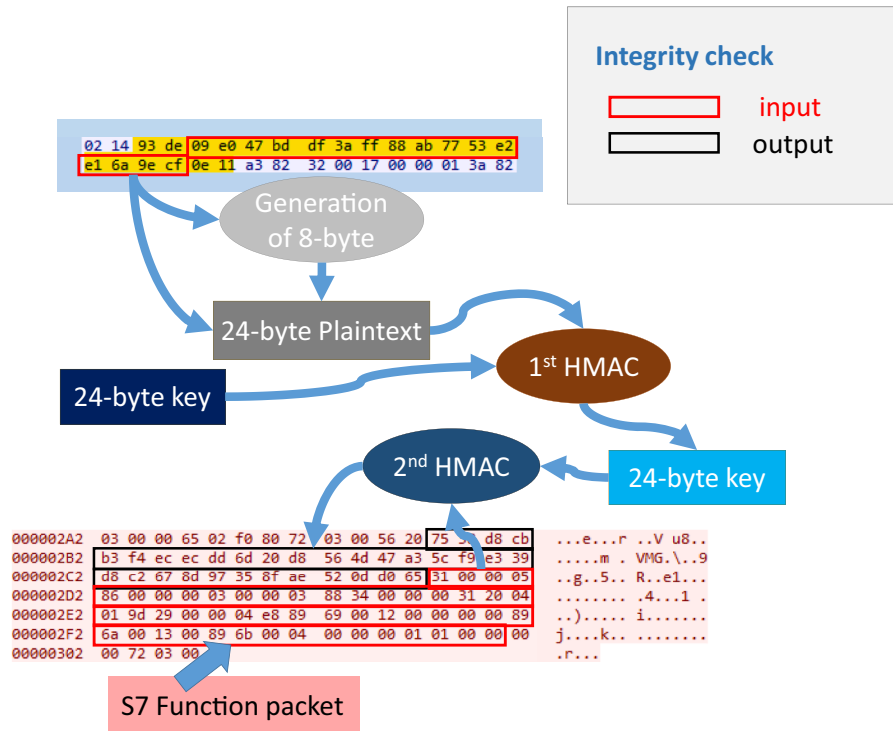


Fig. 14. Integrity check.

```
def gen_8_byte (
    input: 16-byte challenge array, challenge_16
    20 variable length array of 3-byte, b_r8[20][3]
    20 variable length array of constants, b_r12[20][]
    20 fixed length array of lookup byte, b_f0[20][188]
    20 4-byte string, b_xor

    output: 8-byte string)

    counter = 0
    c_array = []

    for round in range(0,20):
        for line in range(0,len(b_r8[round])):
            current_b_r8 = b_r8[round][line]
            byte1 = (( challenge_16[ current_b_r8[0] >> 1] >> (4-4*( current_b_r8[0] % 2)))&0x0f) << 4
            byte2 = ( challenge_16[ current_line[1] >> 1] >> (4-4*( current_b_r8[1] % 2)))&0x0f
            b_r9 = byte1|byte2
            mod47 = (b_r9 >> 3 + counter >> 2) %47
            temp = change_endian(b_f0[round][mod47-4:mod47])
            b_r10 = ((temp ^ b_xor) >> ((7-b_r9&0x07)<<2)) % 0x100
            byte3 = current_b_r8[2]
            shift = 4-4*(byte3%2)
            temp1 = (((b_r12[round][b_r9 >> 1 + counter] >> (4-4*(b_r9%2)))^ b_r10) &0x0f) << shift
            c_1 = (0xf0 >> shift) & challenge_16[byte3 >> 1]
            c_array[byte3 >> 1] = temp1|c_1
            counter += 0x80

    # Reading the first or second half of two byte respectively and concatenate to one byte
    output[0] = concatenate(c_array[0x5d][1],c_array[0xe0][0])
    output[1] = concatenate(c_array[0xbd][0],c_array[0x35][1])
    output[2] = concatenate(c_array[0x77][1],c_array[0x56][1])
    output[3] = concatenate(c_array[0x53][0],c_array[0x21][0])
    output[4] = concatenate(c_array[0xe5][0],c_array[0x3a][1])
    output[5] = concatenate(c_array[0x45][0],c_array[0xa5][1])
    output[6] = concatenate(c_array[0x3f][0],c_array[0x59][0])
    output[7] = concatenate(c_array[0xac][0],c_array[0xf7][1])

    return to_byte_string(output)
```

Fig. 15. Pseudo code for generation of 8-byte block, which is part of the plaintext of the first HMAC.

manipulate the keys without accessing the memory of the PLCs or the machines hosting TIA portal. However, an attacker may be able to use a man-in-the-middle proxy, for example in the form of a TIA portal in a compromised machine, to change the keys to those desired [7]. While the possibility of manipulating the input of the hash is acknowledged by Biham, here we document and visualise (in Fig. 9) how the manipulation of the bytes affects the security of the rest of the mechanism. From a security researcher's perspective this information is important because it reveals the otherwise unknown problems of proprietary protocols and highlights the security limitation of having potentially multiple TIA Portal and PLC instances. As this research was conducted in response to that of Cheng, the presentation and terminology of the analysis is aligned with [6], i.e. identifying the main components as First Encryption, Second Encryption, and the function packet integrity check (described as the third encryption in [6]).

4.2.1.2. First encryption. The two encryptions used in the response packet are first mentioned by Cheng. However, only their position in the IP packet and little other information about the two encryptions is given. Biham [7] subsequently added additional information, documenting the underlying algorithm used to generate these bytes, which is a proprietary version of the Elliptic-Curve El-Gamal key exchange. As a further contribution, we now present the lower-level byte operations and the procedures necessary to manipulate the anti-replay mechanism. As illustrated in Fig. 10, the First Encryption is a 16-byte value located in the 77th to 93rd bytes of the previously identified 132-byte block. It is found that the encryption is done by using the 61st to 76th byte of the 132-byte block, generated in ©, as plaintext and "Key 1" as the encryption key for the symmetric-key encryption algorithm. The output of this encryption will then be XOR-ed with the challenge array, and store in an address that handles communications before sending to the PLC. However, since both the plaintext and the key are related to the manipulated hash in ©, this encryption can be treated as a simple XOR operation of a 16-byte constant and the challenge array.

4.2.1.3. Second encryption. The Second Encryption uses the same encryption algorithm as the First Encryption. However, the plaintext is generated by a more complicated algorithm, which uses the First Encryption as part of the input. An analysis of the complete algorithm is depicted in Fig. 13. Furthermore, a number of building blocks have been identified that are required to generate the plaintext, which are not described by [6] or [7]. These lower level byte operations are presented below in the form of detailed pseudo code. This pseudo code is intended to help with the reproducibility of the presented experiments (and of [6] and [7]).

(1) "Finite field" incrementation algorithm (see Fig. 11). A finite field is a set of elements with a finite order and the operations of multiplication and addition should align with certain rules. The order of a field is the number of the element in a field and the order should be a power of prime.

For this incrementation, the order of the field is 2^{128} , which can be represented by a 16-byte value. During the analysis, it was not possible to find a generalised irreducible polynomial (the rule of operation in the finite field), therefore the computation is represented by a set of operations and a set of constants instead. To begin the operation, a 16-byte value will be multiplied by all integers from 0 to 255. This 16-byte value is generated from the same hash as the two encryption keys (see label ©). It takes the form of four 4-byte little endian values. The computation can be represented as shown in Fig 11. During the multiplication process, if the n^{th} root of the multiplier is 2, where n is a positive integer, i.e. multipliers like 2, 4, 8..., the output will be XOR-ed with a specific value, X_{pn} , and stored in an array with 256 elements, or array "value256[]", where each elements is a 16-byte value. As the value has to stay within the "finite field", the rest of the multiplication cannot simply be achieved by multiplying the value of the input by the

multiplier. The computation is done by first representing the multiplier in its binary form. Then the binary of the multiplier is read from the most significant bit. If it is a '0', there is no further action. But if it is a '1', e.g. if the multiplier is "9", which is "1001" in binary, value256 [2³] and value256[2⁰] will be XOR-ed together and the output is stored in value256 [9].

During the computation, if the value exceeds 16 bytes (128 bits), the first bit of the output will be discarded. After the generation of the 255 values, they will be used in a multiplication-like operation between two 16-byte value as a look-up table.

(2) Algorithm a480. This algorithm, named "algorithm a480" after the memory offset of the address, takes the "value256 []" in Fig. 11, the output of the "finite field" incrementation algorithm, and a 16-byte value as the input. The 16-byte input, "b_input" is divided into four 4-byte array, which are represented in little endian format. Fig. 12 shows the pseudo code for algorithm a480. The first part of the algorithm is executed in a loop. Each of the byte positions of the 4-byte values will be read one after another. Starting by reading from the first byte position in the first element of b_input, to the first byte position in the second element, etc. (i.e. b_input[0][0], then b_input[1][0], etc.). Like any value in hex, it can be 0 × 00 to 0xff, which corresponds to the different 256 elements of the 16-byte value in value256 []. The 16-byte value (the first read value being value256 [b_input [0] [0]]) will be appended by zeros, with the length of zeros depending on the element it is currently on. For example, for element 0, or b_input[0], 16 zeros are appended. Similarly 12, 8, or 4 zeros will be added for b_input[1] to b_input[3] respectively. The result will be stored and be XOR-ed with the next value with added zeros. (e.g. the first two values that are XOR-ed are value256 [b_input [0][0]] and value256 [b_input [1][0]], both with added zeros). Before proceeding to the next byte position (e.g. from b_input[x][0] to b_input[x][1]), each incremental result will be converted into big endian, shifted 1 byte to the right, and be converted back to little endian value again. For the last position, instead of shifting the 32-byte value to the right by 1 byte, a more complicated operation is done. The operations start from the left of the statement in the pseudo code. The operation involves: "<<" binary left shift, ">>" binary right shift, and "^^" bit-wise XOR. Any byte exceeded the 16-byte limit in either direction during shifting will be discarded. The resulting 32-byte value will be divided into 8 different 4-byte values for the operations. At the end of the algorithm a 16-byte string is obtained by concatenating the four 4-byte array into a byte string.

(3) Plaintext generation. As shown in Fig. 13, there are five different inputs in the plaintext generation process for the Second Encryption. The five inputs are: 1) a 16-byte value; 2) the result of the First Encryption (red box in Fig. 13); 3) a 16-byte ciphertext (green box in Fig. 13); 4) an 8-byte value curtailed from another ciphertext (blue box in Fig 13); and 5) a 4-byte counter value padded with zeros. Experiments have shown that all of these inputs, except the result of the First Encryption, are constant relative to the two hashes identified at point © in Fig. 9. Each step of the plaintext generation process involves XOR-ing two inputs and running the result through the "algorithm a480".

(4) Finalising Second Encryption. After the plaintext is generated, it is encrypted using "Key 2" (see Fig. 9), using the same symmetric-key encryption as the First Encryption. The ciphertext will be used as the last 16-byte of the 132-byte block, i.e. the anti-replay response from TIA Portal. Fig. 13 illustrates how the "finite field" incrementation algorithm, algorithm a480, and the plaintext generation are related to each other. The byte position of the input in the 132-byte array, and how the Second Encryption is generated can be found in the figure as well. Compared to Cheng's work, a more complete description is provided on the generation of the S7 response packet, with details of the two encryptions also providing extra information beyond the later work by Biham. Notably, the algorithm a480 has previously been briefly presented as the tabulation hash, but without details around the steps for generating the second encryption. This further analysis is presented

now, due to the previously described lower level byte computation. This allows an initial proposal towards patching the possible manipulation. The underlying problem of the mechanism will be discussed in Section V.

4.2.2. Function packet from TIA portal

After the S7 challenge and response packets are exchanged, S7 function packets will then be sent. These packets include the purpose and the detail of the communication and Fig. 8 shows one of the packets which contains control information sent by TIA Portal in the example given in Section III.B. Every packet has to include the 32-byte “Encryption” (as called by Cheng) before the payload, as shown by the outlined byte in the same figure. It is found in this analysis that this 32-byte block is in fact an HMAC integrity check for the packet. This integrity check serves two purposes: making sure the payloads are not manipulated; and authenticating the sender (since the keys for the HMAC are known only by the hosts that are involved in the connection). In order to generate this 32-byte value, two HMAC computations were found. The first is used to generate the key for all subsequent HMACs. The second is used to sign all the function packets. Both HMACs are based on the same hashing algorithm as the rest of the mechanism. Fig. 14 shows how both HMACs generate the integrity bytes.

4.2.2.4. First HMAC. The computation of the first HMAC is done before the S7 Response packet is sent to the PLC. The plaintext of the HMAC consists of an 8-byte value and the 16-byte challenge array, read from breakpoint C (see Fig. 9). Fig. 15 shows pseudo code for the 8-byte generation, which is critical to the S7 function integrity check, but not previously described in the literature. This algorithm is not identified as a standardised algorithm, therefore the pseudo code is generated by an analysis of the assembly code. This analysis is instructive from a security perspective, as the 8-byte generation algorithm is an essential part for the integrity byte generation. The combined 24-byte value is then signed using a 24-byte key generated in the black-box at label ⑥ in Fig. 9. The 32-byte output of the HMAC will be curtailed to a 24-byte value, which will be stored and used as the key of the second HMAC.

4.2.2.5. Second HMAC. The second HMAC is used to generate the actual integrity check bytes, which are inserted in the function packet sent by both parties. It has been identified here for the first time how the key of the second HMAC is the result of the first HMAC, and furthermore which part of the S7 function packet payload is used as the input of the second HMAC. The length of the function packet is not always the same, but the 32-byte HMAC in the packet always starts at the 13th byte of the packet. The input of this HMAC comprises all the bytes following the HMAC in the packet, i.e. starting from 45th byte, excluding the footer of the packet which is usually the last four bytes (for example in the case of Fig. 8 it is “72 03 00 00” at the end of the packet). As the length of each packet and the information contained can vary, the length and content of the footer vary accordingly. However, in order to replay the packet, since the key is known, a simple trial and error method could identify what byte is needed as the input of the HMAC.

4.2.3. Summary and discussion

This section documented the methods and experimental process required to obtain information about the anti-replay mechanism used in the S7CommPlus protocol. This newly found information, including the standardised hashing and encryption algorithm used, and the proprietary algorithm for plaintext and key generation, can potentially be exploited to create tools to generate “legitimate” S7 communications. This will be investigated further in Section V.

Based on the above experiments, a number of interesting practical aspects will now be discussed, with the aim to provide future researchers in this area with guidance to help reproduce similar experimental outcomes.

Although not utilised during this investigation, “IDA PRO” or the new open software “Ghidra” may significantly speed up an investigation, especially to discover where specific functions are used in various parts of the software.

Where a challenge and response mechanism are present, following or back-tracing the memory location of the challenge in the memory can provide an entry point for the reverse engineering process.

Logging the output of the “ta” (trace to address) command in Windbg provides a way to find where specific bytes are generated. Although not utilised in this work, custom scripts may be useful to dump extra information during the trace.

Most of the major breakthroughs during this investigation came when it was discovered that various bytes used or stored in the memory are constants used by cryptographic functions. For example, the s-box of AES and initial hash value of sha-256. Therefore, when performing similar analysis, searching for cryptology constant that is used in the memory may provide a simple way to identify a specific algorithm when only the assembly code is available.

After a standard algorithm is found to be used, for example AES, searches in the memory and back-tracing to the parent function may provide information revealing multiple instances where the algorithm is utilised.

If a cryptographic key is generated solely from one side of the communication, without input from another party, the key may be manipulated.

5. Exploits and mitigation

5.1. Potential exploits

The above analysis of the S7CommPlus protocol and TIA Portal reveal the possibility of exploiting the protocol and the software. Several validated exploits are discussed below, as are potential additional attacks that may be pursued by an attacker sufficiently motivated to further analyse the discoveries presented in Section IV and to proceed to develop malicious exploits.

5.1.1. Unauthorised changes to PLC state

Since all actions performed on TIA Portal are sent to the PLC using the S7 protocol, by utilising the discoveries in this work it may be possible to cause disruption to a number of processes, including: reprogramming a PLC, changing a PLC variable value, setting password to a PLC, and changing the state of PLC (from run state to stop state, and vice versa). For demonstration purposes, the scope of the current work has focused on whether it is possible to remotely change the state of a PLC, i.e. stop a running PLC. Experiments have demonstrated that this can indeed be achieved by using two S7 function packets in addition to the anti-replay response, which includes the 132-bytes identified in Fig. 7 at point ⑧. One of the spoofed function packets that is required to stop a PLC is shown in Fig. 16. The spoofed packet is generated based on the discoveries documented in section IV. Other than the anti-replay byte (④ in Fig. 6), the integrity check bytes and the sequence number (not related to the anti-replay mechanism), all other bytes are the same in both of the packets in Fig. 16 (a normal and a spoofed packet). It is worth mentioning again that even in different S7 sessions, all the anti-replay bytes can be generated for any spoofed packet based on the same encryption keys and required bytes that will remain constant based on the hashes in ⑩ and black-boxes ③ and ⑤ in Fig. 9.

As mentioned, it is also possible to reprogram a PLC using the S7 protocol. Although there is a copy protection mechanism built in, as long as attacker can identify the relationship between the serial number of the PLC that is sent along the PLC logic and the PLC logic itself [13], an attack to reprogram the logic may be feasible. However, for now this was considered out of scope for the analysis presented in this paper.

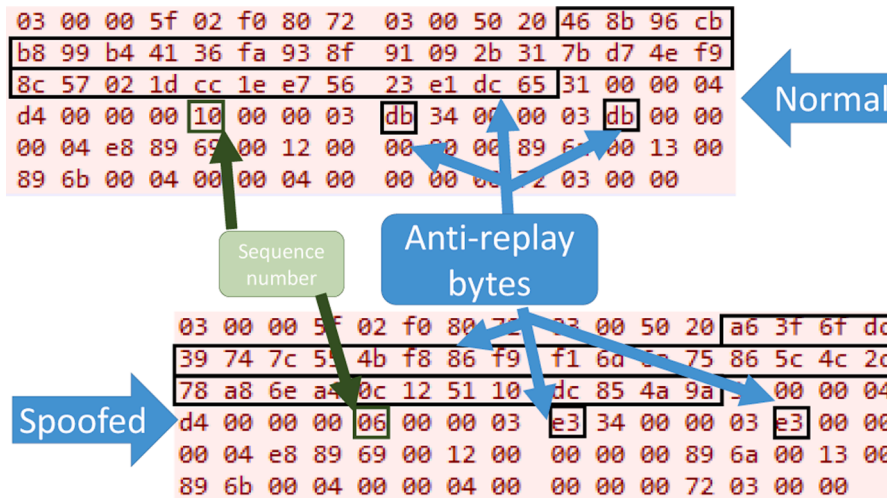


Fig. 16. Normal vs Spoofed traffic.

5.1.2. S7 communication DOS

Other than exploiting the normal functionality provided via TIA Portal, the findings from Section IV have been shown to enable a DOS attack by sending crafted packets which establish and maintain a new S7 session to a PLC. This will prevent TIA portal from connecting to the PLC. This exploit is possible because the S7-1211C will not allow a new session to be initiated if there is already an existing session. To apply this exploit it is assumed that there is no current session between the PLC and TIA Portal. A host on the same LAN as the PLC can initiate a TCP connection to the PLC via the usual handshake and exchange of COTP packets. By responding to the challenge packet from the PLC with a crafted packet containing the appropriate anti-replay bytes, followed by “S7-ACK” packets, the attacker can prevent a genuine connection from TIA Portal. In order to keep a normal session alive, for example to ask the other end to wait while a process executes, either end connection can reply with such an S7-ACK packet, seemingly indefinitely [19]. This exploit is possible because the “S7-ACK” packet, “03 00 00 07 02 F0 00”, lacks anti-replay or integrity check functions, and can be used to reply to any S7 packets. Therefore, the session can be kept alive without the attacker obtaining any extra information. Although the PLC will continue to run the pre-programmed logic, it is not possible to stop, re-configure or re-program the PLC. A manual restart can terminate the existing session, however a compromised host or device in the network could re-initiate a new DOS session. This attack could be critical on its own, but it could also enable a larger scale attack.

5.1.3. Session hijacking

The exploit described above assumes no existing connection between TIA Portal and the targeted PLC. However, this exploit could be combined with a traditional network attack, using crafted exploit packets along with ARP poisoning, to hijack a session to the PLC and cause a DOS so TIA Portal cannot reconnect. The authors have previously documented how session stealing by ARP poisoning can work in this context [19]. This can be achieved by actively dropping all packets from the engineering software after stealing the S7 session, which causes the TIA Portal side of the connection to be terminated. At the same time, the PLC side of the session can be kept alive by an attacker crafting and sending S7-ACK packets. During such an attack, one option is to use excessive ARP replies to overwhelm any other ARP packets after the session is stolen. However, to avoid generating too much ARP “noise”, the stolen S7 session could be terminated and replaced by a new DOS session, as described in the previous section. Fig. 17 illustrates the steps taken to hijack a legitimate S7 session. Two points are achieved from this attack: 1) A smaller network “footprint” would be generated when compared with only utilising ARP poisoning session stealing. 2) It is possible to

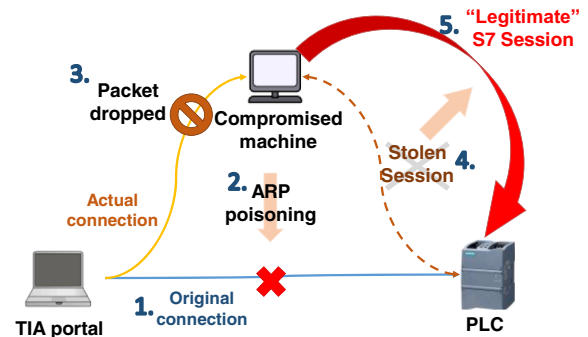


Fig. 17. Session hijacking.

perform DOS attack without waiting for the legitimate session to terminate, as a DOS session also cannot be started if a legitimate one exists.

5.1.4. Main program removal

A new exploit based on the S7CommPlus protocol that can manipulate the PLC logic has been found and is now presented.

This exploit can remove the Main Object block of the S7-1200 PLCs and causes the TIA portal to provide incomplete or incorrect information of the PLCs. It can be produced by replaying S7 packets that are captured under an abnormal use case in the TIA Portal. The steps of reproduction of the exploit are described below including the packet generation, replaying of the packets and the expected behaviour.

Starting from the generation of the packets that will be replayed as the exploit:

- 1 Create a new project in TIA portal and add a PLC to the project with the firmware version of the target PLC (which can be obtained by the LLDP broadcast from the PLC or by starting a session with the PLC).
- 2 Upload a default project (which has only an empty Main object block) to the testing PLC (a PLC that has the same firmware version as the target PLC).
- 3 Making sure the PLC is in the Stop state.
- 4 Delete the Main object block in the project and start a network capture, and then upload the project with the option “Software (changes)”. End the capture after the PLC end the session using a TCP packet with Reset and Finish flags set.
- 5 Extract all the packet sent by the TIA portal for the use in the script, which was sent in the previous communication.

Sending (replaying) these captured packets to a target PLC, which have the same firmware version, that is in the stop state, will cause abnormal behaviours. Packets captured from different firmware, will cause different behaviours on a different firmware version. This exploit was tested on S7-1211C DC/DC/DC, firmware versions v4.1.3 and v.4.2.3.

For v4.1.3, the capture can be replayed as it is, and the PLC will reset the connection after the crafted packets are sent to the PLCs. The exploit may have the following behaviours, depending on the settings and status of the PLC:

- i When going online to the infected PLC, the PLC cannot be set to the run state. If a user is trying to download the custom code from TIA portal to the PLC, the TIA portal will indicate that the program between the PLC and the TIA portal are the same with no error shown. However, the PLC is running with no Main Object Block, i.e., performing no action. A possible fix is to go offline and download the complete configurations to the PLC using the download option “Hardware and Software (only changes)”. A window for synchronising program will be shown before downloading, showing there are differences between the logic in the PLC and the project in the TIA portal.
- ii When going online, the PLC cannot be set to the run state. If a user trying to download the custom code to the PLC, a window for synchronisation will be shown before downloading. In addition, either no error will be shown or an error saying “Main object block does not exist” will be shown.
- iii When going online, PLC can be set to the run state. However, in reality no logic will be running in the PLC, and all the program blocks are shown as “only exist offline” in the TIA portal. Synchronisation is required before downloading.

If the packets captured from a PLC using v4.1.3 firmware are replayed to the PLC running v4.2.3, scenario iii will happen.

When the packets are generated by v4.2.3 PLCs and applied to the PLC with the same firmware, the replay attack is likely to cause scenario iii described above. Further misbehaviour is observed when the replay is cut short, i.e. not replaying the entire captures, the PLC will behave as follows:

The user can set the PLC to the run state. However, no logic in the Main object block will be running, but Cyclic interrupt or Hardware interrupt may continue running. If the user tries to turn on “monitoring” in the Main object block while online (an option in the TIA portal that provides diagnostic information for the PLCs in real time), the error “Internal system error (error code:0xCF000AF0700008002)... Internal error: A referenced FunctionalObject does not exist.” will be shown. A possible fix is to download the project to the PLC, and manual synchronisation of the custom program will be required.

During testing, there are two occasions that the PLC went into an unrecoverable state after various captures were replayed to the device repeatedly. The only feasible fix in this situation is a factory reset. Unfortunately, captures of the exact packet sequences required were not kept, as the behaviour seems to be somewhat random and unpredictable.

5.2. Related work and password protection

Responses from Siemens, to both of the closest related work [6,7] and this work is the recommendation on enabling password protection for all PLCs. However, conversation with an industry representative reveal that in practise this is rarely exercised, or weak password are used. Often the passwords are also shown directly in front of the PLCs. This is partly due to a lack of password management scheme to a network of PLCs.

Regardless, further experiments were conducted to establish the protection offered in the case when the PLC is password protected. Table 3 shows the comparison of the use of the password protection against the exploits proposed in the previous sections. Actions available in the TIA Portal, e.g. stopping a PLC, will be protected with the use of the password. However, the other exploits that make use of the S7-ACK packet as a DOS attack, are still feasible despite a password being used. This is because the S7-ACK can be sent by TIA Portal, or an attacker, at any time in response to an S7 packet.

Both this work and [7] described the detail workings of the integrity check using in the S7CommPlus protocol, although tested on S7-1200 PLCs and S7-1500 PLCs respectively. However, this work has presented a greater level of details on the rest of the protocols, like the key generation manipulation, and details of the algorithms used during the anti-replay byte generation.

5.3. Mitigation

In previous research, Cheng [6] recommended that Siemens provide code obfuscation techniques on the key dll files. However, this would only be effective to a certain degree as it is providing security by obscurity, which itself is not a robust security measure. A sophisticated process of reverse engineering, like the one presented in section IV, would still be possible. Public key cryptography would be beneficial, but the computation cost would be increased and therefore only more advance PLCs would have the resources to perform the computation. On the other hand, Biham suggested that PLCs refuse any TIA session that is not previously authenticated. This perhaps improves the security of the PLC operations, however, a new workflow for practical operations would be required, especially for scenarios where an engineer may bring in a laptop for PLC configuration, and credentials would have to be carefully managed.

Spennenberg [10] outlined the protection mechanism in place for older PLCs from Siemens, e.g. password protection, copy protection, and access protection. Although these protection mechanisms remain available in newer firmware versions and do protect against some of the attacks (e.g. stopping a PLC) they do not stop attackers analysing TIA Portal to discover how these mechanisms work. Moreover, the attacks mentioned in Section V.A.1) and V.A.2) will still be feasible. Furthermore, password protection cannot protect against all the exploits discussed in the paper, as discussed above.

During the analysis of TIA Portal, some standardised cryptographic algorithms were identified. However, the way the algorithms are applied are suspected to be non-standard. To some extent this can provide security by obscurity, but it may also invite further investigation by a

Table 3
Feasibility of exploits with and without password, and comparison with most recent published work.

	This work With password protection	Without password protection	Cheng[6] With password protection	Without password protection	Biham[7] With password protection	Without password protection
Performing actions available on TIA Portal (Section 5.1.1)	✗	✓	✗	✗	✗	✓
DoS on S7 Communication (Section 5.1.2)	✓	✓	✗	✓	✗	✓
Session Hijacking (Section 5.1.3)	✓	✓	✗	✓	✗	✗
Main Program Removal (Section 5.1.4)	✗	✓	✗	✗	✗	✓

dedicated and motivated adversary.

5.3.1. Recommendation for the vendor

As explained in Section IV.B, at least two hashes play a significant role in the anti-replay mechanism. It was found that the input of these hashes could be manipulated by amending the memory used by the software. This vulnerability allows an attacker to generate a S7 session requiring only 17 of the 150 anti-replay bytes in the S7CommPlus response packet (the Second Encryption and the one anti-replay byte) as opposed to 150 bytes (the 132-byte, 8-byte, and 9-byte blocks, plus the anti-replay byte). Furthermore, all the keys involved, either for the First Encryption and Second Encryption, or the HMACs in the function packets, stay constant relative to the input of the hashes. These two issues significantly reduce the possible security provided by the algorithm, as well as seemingly wasting resources. In the short term, adoption of the below proposed security measures could limit the ability of an attacker to perform similar exploits, without significant design changes.

5.3.1.6. Change the input of the hash algorithm. Some bytes exchanged between TIA Portal and a PLC are not used, nor is there an obvious impact for those bytes on the communication. Consider the following discoveries:

- 1 Server session and PC session, labelled ① and ② in Fig. 5, appear to serve no purpose in the protocol and can be reused in different sessions, which defeats the purpose of a session number.
- 2 It has been shown that the 16-byte “challenge array” in the 20-byte challenge sent by the PLC plays an important role in the anti-replay mechanism. However, the remaining 4 bytes were not identified to make a difference in the anti-replay mechanism.
- 3 In the S7 response packet from TIA portal, the bytes labelled ⑥ and ⑦ in Fig. 7, were generated before the 132-byte value, label ③. However, ⑥ and ⑦ stay constant with respect to the two hashes and no relationship to the anti-replay mechanism is discovered.

Consequently, it is proposed that one option to limit an attacker’s ability to generate a viable S7 packet is to include some the bytes mentioned above into the anti-replay mechanism. For example, it is suggested to include the server session and the four remaining bytes of the challenge array into the hash algorithm that generates the encryption keys, along with a brief history of the challenge array as part of the input to generate the anti-replay bytes. This will not remediate the threat of an attacker using reverse engineering techniques on the software to obtain the input and the algorithm used in the anti-replay mechanism, but anyone attempting this will need to generate the entire 150 anti-replay bytes, including determining the keys in every attack, and be able to obtain previous sessions of TIA Portal communication.

5.3.1.7. Limit the session lifetime. During the analysis, it was discovered that a TCP session will stay alive until TIA Portal either terminates the communication, fails to acknowledge TCP keep alive packets or S7 packets sent by the PLC, or sends the incorrect anti-replay bytes. For example, if a breakpoint is set on TIA Portal during the communication with the PLC, legitimate S7CommPlus packets will no longer be sent. Meanwhile the PLC will send periodic TCP keep alive packets and will not terminate the communication as long as TIA Portal replies with the TCP keep alive packet. This increases the time available for any prospective attacker to perform a more in depth analysis at each stage of the communication. Moreover, the S7-ACK packet can be exploited to perform a denial of service attack, through resource exhaustion, since the amount of resources dedicated to communication in a PLC is limited.

A potential mitigation would be an update to the PLC firmware to ensure PLCs will disconnect from an idle S7 session (a session containing

only S7-ACK packets) or after not receiving any legitimate S7 packet after a given time period, unless the PLC is otherwise configured by the operator.

6. Conclusion

This paper presents a security analysis of the anti-replay mechanism used in the S7CommPlus protocol, with experiments validating the findings using the Siemens S7-1211C PLC, firmware v4.1 and v4.2, and TIA Portal v14, are documented. It has been shown that an attacker with network access can send packets to a PLC, read a challenge from the PLC (first adopted in firmware v4), calculate the response, and create integrity checks in subsequent packets to maintain a valid session. This allows an attacker to craft packets that will be accepted by the PLC, for example to cause a PLC to stop, start CPU logic that has previously been stopped by a legitimate user, hijack an existing legitimate session between a PLC and TIA Portal (demonstrated to cause a DOS), and potentially other issues such as reprogramming the CPU that are of interest for future exploration. Compared to the work by Cheng [6], this work provides a more detailed analysis describing the anti-replay mechanism of S7CommPlus, including new information about the two symmetric key encryptions involved in the 132-byte block and the HMAC in the function packets. On the other hand, new insight is provided in this work regarding the anti-replay byte generation, including the details of the algorithms used and the way to manipulate the keys being generated. As Table 3 highlights, [6] and [7] only demonstrated exploits allowed by TIA Portal without the use of a password, whereas several of the now presented methods are possible despite the password being enabled. For example, it was found in this work that the S7-ACK packet, which does not require integrity or authentication bytes in the packet, has the potential to be exploited, regardless of whether the PLC is password protected or not. In addition to this new information about the anti-replay mechanism, a novel approach to allow manipulation of the cryptographic keys used in the anti-replay mechanism is also presented. Consequently, the discoveries made during these experiments prove an attacker can replay S7 packets by generating the anti-replay bytes using the manipulated keys. The feasibility of this approach raises further questions about the potential for an attacker to replay any legitimate S7 packets that support the TIA Portal functions. In the meantime, the authors have proposed a number of new mitigation steps that have not previously been proposed in related research. These steps may be undertaken to make such exploits more difficult for a determined attacker to discover and execute and; namely, to adopt a timeout approach to mitigate potential DOS, and to change the way the hash algorithm is implemented to ensure the manipulation of TIA Portal functions is more difficult for an attacker to achieve. Similar to cyber security research in other fields, the authors hope the provided information on performing this vulnerability analysis will provide the research community a deeper insight towards understanding attacker approaches and viable mitigation strategies.

Declaration of Competing Interest

None.

References

- [1] J.H. Park, M. Kim, D. Kwon, Security weakness in the smart grid key distribution scheme proposed by Xia and Wang, *IEEE Trans. Smart Grid* 4 (3) (2013) 1613–1614.
- [2] Y.-M. Kwon, J. Yu, B.-M. Cho, Y. Eun, K.-J. Park, Empirical analysis of mavlink protocol vulnerability for attacking unmanned aerial vehicles, *IEEE Access* 6 (2018) 43203–43212.
- [3] M. Kim, J. Lee, J. Paek, Neutralizing BLE beacon-based electronic attendance system using signal imitation attack, *IEEE Access* 6 (2018) 77921–77930.
- [4] L. Martín-Liras, M.A. Prada, J.J. Fuertes, A. Morán, S. Alonso, M. Domínguez, Cybersecurity comparative of configuration protocols in industrial control devices,

- Int. J. Crit. Infrastruct. Prot. 19 (2017) 4–15, <https://doi.org/10.1016/j.ijcip.2017.10.001>.
- [5] A. Ghaleb, S. Zhioua, A. Almulhem, On PLC network security, *Int. J. Crit. Infrastruct. Prot.* 22 (2018) 62–69.
 - [6] L. Cheng, L. Donghong, and M. Liang, “The spear to break the security wall of S7CommPlus,” 2017, [Online]. Available: <https://media.defcon.org/DEFCON25/DEFCON25presentations/ChengLei/DEFCON-25-Cheng-Lei-The-Spear-to-Break-the-Security-Wall-of-S7CommPlus-WP.pdf>.
 - [7] E. Biham, S. Bitan, A. Carmel, A. Dankner, U. Malin, A. Wool, Rogue7: rogue engineering-station attacks on S7 Simatic PLCs, Black Hat USA (2019).
 - [8] N. Falliere, L.O. Murchu, E. Chien, W32.Stuxnet Dossier, Symantec-Security Response (2011) 1–69, vol. Version 1., no. February 2011doi: 20 September 2015.
 - [9] R. Langner, Stuxnet: dissecting a cyberwarfare weapon, *IEEE Secur. Priv.* 9 (3) (2011) 49–51, <https://doi.org/10.1109/MSP.2011.67>.
 - [10] International Electrotechnical Commission, IEC 61131-3:2013 programmable controllers - Part 3: programming languages, Int. Standard (2013). <https://webstore.iec.ch/publication/4552> (accessed Feb. 05, 2018).
 - [11] D. Beresford, “Exploiting Siemens Simatic S7 PLCs,” 2011, Accessed: Apr. 03, 2017. [Online]. Available: https://media.blackhat.com/bh-us-11/Beresford/BH_US11_Beresford_S7_PLCs_WP.pdf.
 - [12] J. Klick, S. Lau, D. Marzin, J.O. Malchow, V. Roth, Internet-facing PLCs as a network backdoor, in: 2015 IEEE Conference on Communications and Network Security, CNS 2015, 2015, pp. 524–532, <https://doi.org/10.1109/CNS.2015.7346865>, no. SPiCy.
 - [13] R. Spennenberg, “PLC-Blaster: a Worm Living Solely in the PLC,” 2016, [Online]. Available: <https://www.blackhat.com/docs/us-16/materials/us-16-Spenneberg-PLC-Blaster-A-Worm-Living-Solely-In-The-PLC-wp.pdf>.
 - [14] H. Wardak, S. Zhioua, A. Almulhem, PLC access control: a security analysis, in: 2016 World Congress on Industrial Control Systems Security (WCICSS), 2016, pp. 1–6, <https://doi.org/10.1109/WCICSS.2016.7882935>.
 - [15] G.P.H. Sandaruwan, P.S. Ranaweera, V.A. Oleschchuk, PLC security and critical infrastructure protection, in: 2013 IEEE 8th International Conference on Industrial and Information Systems, 2013, pp. 81–85, <https://doi.org/10.1109/ICIInfS.2013.6731959>.
 - [16] B. Lim, D. Chen, Y. An, Z. Kalbarczyk, R. Iyer, Attack induced common-mode failures on PLC-based safety system in a nuclear power plant: practical experience report, in: 2017 IEEE 22nd Pacific Rim International Symposium on Dependable Computing (PRDC), 2017, pp. 205–210, <https://doi.org/10.1109/PRDC.2017.34>.
 - [17] X. Lv, Y. Xie, X. Zhu, L. Ren, A technique for bytecode decompilation of PLC program, in: 2017 IEEE 2nd Advanced Information Technology, Electronic and Automation Control Conference (IAEAC), 2017, pp. 252–257, <https://doi.org/10.1109/IAEAC.2017.8054016>.
 - [18] Industry Support Siemens, “Announcement: product Phase-Out for SIMATIC S7-200 - ID: 67598674,” 2013. <https://support.industry.siemens.com/cs/document/67598674/announcement%3A-product-phase-out-for-simatic-s7-200?dti=0&lc=en-WW> (accessed Mar. 19, 2018).
 - [19] H. Hui, K. McLaughlin, Investigating current plc security issues regarding siemens s7 communications and TIA portal, in: 5th International Symposium for ICS & SCADA Cyber Security Research, 2018, pp. 67–73.
 - [20] S. McLaughlin, S. Zonouz, D. Pohly, P. Mcdaniel, A trusted safety verifier for process controller code, in: *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2014, pp. 23–26, no. February.
 - [21] S. Mohan, S. Bak, E. Betti, H. Yun, S3A: secure system simplex architecture for enhanced security of cyber-physical systems, in: *Proceedings of the 2nd ACM international conference on High confidence networked systems*, 2013, pp. 65–74, <https://doi.org/10.1145/2461446.2461456>.
 - [22] S. McLaughlin, CPS: stateful policy enforcement for control system device usage, in: *ACSAC '13 Proc. 29th Annu. Comput. Secur. Appl. Conf.*, 2013, pp. 109–118, <https://doi.org/10.1145/2523649.2523673>.
 - [23] T. Cruz, et al., Improving network security monitoring for industrial control systems, in: 2015 IFIP/IEEE Int. Symp. Integr. Netw. Manag, 2015, pp. 878–881, <https://doi.org/10.1109/INM.2015.7140399>, no. May.
 - [24] T. Cruz, et al., A cybersecurity detection framework for supervisory control and data acquisition systems, *Ieee Trans. Ind. Informatics* 12 (6) (2016) 2236–2246, <https://doi.org/10.1109/TII.2016.2599841>.
 - [25] S. Ponomarev, T. Atkison, Industrial control system network intrusion detection by telemetry analysis, *IEEE Trans. Dependable Secur. Comput.* 13 (2) (2016) 252–260, <https://doi.org/10.1109/TDSC.2015.2443793>.
 - [26] P. Kreimel, O. Eigner, P. Tavaloto, Anomaly-based detection and classification of attacks in cyber-physical systems, in: *Proceedings of the 12th International Conference on Availability, Reliability and Security - ARES '17*, 2017, pp. 1–6, <https://doi.org/10.1145/3098954.3103155>.
 - [27] W. Jardine, S. Frey, B. Green, A. Rashid, SENAMI, in: *Proceedings of the 2nd ACM Workshop on Cyber-Physical Systems Security and Privacy - CPS-SPC '16*, 2016, pp. 23–34, <https://doi.org/10.1145/2994487.2994496>.
 - [28] G. Combs, “Wireshark.” <https://www.wireshark.org/>.
 - [29] Philippe Biondi, “Scapy.” 2017, [Online]. Available: <http://secdev.org/projects/scapy/>.
 - [30] Microsoft, “WinDbg.” <https://docs.microsoft.com/en-gb/windows-hardware/drivers/debugger/>.
 - [31] M.T. Rose, D.E. Cass, RFC 1006: ISO transport services on top of the TCP: version 3, Northrop Res. Technol. Cent. (1987).